Remote access to LAN via NetBird - A ring to rule them all

guillaume@labtoforge.fr - www.labtoforge.fr

Version 1.0 - 2026-10-29



- Remote access to LAN via NetBird - A ring to rule them all
  - Inventory and requirements
    - Inventory
    - Requirements
  - Part I - Brainstorming, comparison and decisions
    - I.1 Candidate solutions - what each actually is
    - I.2 Full feature comparison
    - I.3 Selection though process
    - I.4 Security vs comfort - hosting the NetBird controller as a VM *on* the Freebox
    - I.5 Setup steps overview (what follows, at a glance)
  - Part II - NetBird Controller setup (Ubuntu VM on the Freebox)
    - II.1 Infrastructure requirements (official)
    - II.2 Create the VM on the Freebox
    - II.3 Networking / port-forwarding
    - II.4 DNS (OVH zone)
    - II.5 Ubuntu OS hardening and baseline setup
    - II.6 Run the quickstart script
    - II.7 First login
  - Part III - NetBird Groups and Access Policies
    - III.1 Why Peer-to-peer network, not “remote access network” (Networks/routing-peer model)
    - III.2 Core concepts (official)
    - III.3 Recommended approach for this setup
    - III.4 Install and enroll NetBird (Timon hosts on Windows)
    - III.5 Install and enroll NetBird on the iPhone
    - III.6 NetBird Custom DNS Zone
  - Part IV - Timon and Windows reachable via NetBird
    - Non-negotiables
    - IV.1 Windows background-run notes
    - IV.2 Keep Timon’s listeners on loopback
    - IV.3 OVH API credentials setup for DNS-01
    - IV.4 Install Caddy as a Windows service with `sc.exe` (built into Windows, no extra tool)
    - IV.5 Apply and use
    - IV.6 Verify
    - IV.7 Access control reminder
  - Conclusion: Architecture overview
    - Why external traffic reaches LAN devices despite the Freebox blocking incoming connections
    - Connection flow, step by step
  - Glossary

A single complete document - the entire journey plus the full setup to install and configure my own self hosted NetBird and prepare Timon's instances to peer them all together.

I am not a Linux expert, neither a network guru, so trivial "hardening" rules may look insufficient for tenured cable guys. I'll review this documentation in the future to strengthen the rules and improve the hardening.

Inventory and requirements

Inventory

- Free as the internet provider, a Freebox Ultra as the home entry point.
- A home network with wireless and ethernet connected devices.
- A combination of laptops, desktops and NUCs to host home automation, Timon's instances and work-related stations.
- A bunch of IoT devices, either connected on the LAN or with Z-Wave/Zigbee protocols.
- OVH as the custom domain provider.

Requirements

- Secure remote access, from anywhere, to specific devices on the home LAN - the machine(s) running **Timon** and the **home-automation hub**.
- Access must also work **from an iPhone**, over cellular, not just from another LAN-connected laptop.
- No external direct access to IoT and out-of-scope devices.
- Priorities, in the order given: **(1)** secure, at most, your own devices and network; **(2)** maximum visibility into the components involved; **(3)** trustable code end-to-end; **(4)** reduce setup/maintenance effort.
- Candidates to evaluate: **Tailscale**, **Headscale**, **Netbird** - plus whatever else seemed relevant (plain WireGuard was added as a baseline/fourth option during the comparison).

Part I - Brainstorming, comparison and decisions

I.1 Candidate solutions - what each actually is

- **Tailscale**: commercial VPN mesh product built on WireGuard. Client is open source; the **coordination/control server is proprietary SaaS** run by Tailscale Inc. Devices register with Tailscale's servers, which distribute keys/routes (NAT traversal via DERP relays when direct UDP fails). Known for extremely low setup effort and polish (`tailscale serve`/`tailscale funnel` for one-command HTTPS publishing, MagicDNS, excellent mobile apps).
- **Headscale**: open-source, community-built reimplementation of Tailscale's control-plane protocol, meant to be a drop-in self-hosted replacement so you can keep using official Tailscale clients while running your own coordination server. Explicitly described by its own maintainers as reverse-engineered and not officially supported/guaranteed by Tailscale - feature parity lags (e.g. historically slower to support new ACL/tagging features, SSH features, funnel equivalents).
- **Netbird**: fully open-source project (client, management server, signal server, relay/TURN, dashboard) built around WireGuard, explicitly designed to be **either** cloud-hosted (NetBird Cloud) **or self-hosted** as a first-class, officially supported deployment path (not a side reimplementation of someone else's closed system). Includes its own groups/ACL/policy model, private DNS (Custom Zones), and a "Networks" feature for routing non-client-capable devices.

- **Plain WireGuard** (no control plane at all): just the kernel/userspace WireGuard protocol implementation, configured by hand - `wg genkey`, manual `[Peer]` sections, manual NAT traversal/port-forwarding since there's no relay/STUN service, no dashboard, no groups, no revocation UX beyond manually editing config files and restarting the interface.

I.2 Full feature comparison

Criterion	Tailscale	Headscale	Netbird (self-hosted)	Plain WireGuard
Underlying crypto/data plane	WireGuard	WireGuard (via Tailscale client)	WireGuard	WireGuard
Control plane openness	Proprietary SaaS	Open source, reverse-engineered reimplementation	Open source, officially self-hostable	N/A (no control plane)
Self-hosting officially supported	No	Unofficial/community	Yes, first-class	N/A
NAT traversal / relay	Built-in (DERP)	Built-in (reuses Tailscale's DERP infra or self-hosted DERP)	Built-in (own signal + relay/TURN)	Manual (you must port-forward or use an external STUN/TURN yourself)
ACL / groups model	Yes (tags + ACL JSON)	Yes (mirrors Tailscale's model, partial parity)	Yes (groups + policies, more visual/dashboard-driven)	None (hand-edited peer list)
Private DNS for services	MagicDNS	MagicDNS (reimplemented)	Custom Zones (per-group DNS)	None (manual `/etc/hosts` or external DNS)
One-command HTTPS publish	Yes (`tailscale serve`/`funnel`)	Partial/evolving	No native equivalent - manual reverse proxy required	No
Mobile apps (iPhone)	Excellent, official	Uses official Tailscale app pointed at custom control server	Official NetBird iOS app, solid but less polished than Tailscale's	None (would need a generic WireGuard app + manual config/QR)
Visibility into full stack	Partial (control plane is opaque SaaS)	Full (if you self-host) but running someone's reverse-engineering of a closed protocol	Full - every component you run is the official, tested implementation	Full, but minimal - nothing to see because there's no management layer
Setup effort	Lowest	Medium (deploying Headscale + keeping up with Tailscale protocol changes)	Medium (self-host script, DNS, port-forwarding, reverse proxy for app publishing)	Highest (everything manual, scales badly past a few peers)
Maintenance effort	Lowest (fully managed)	Medium-high (track Tailscale client compatibility)	Medium (Docker Compose stack to patch/update; peer/policy admin via dashboard)	High (manual key rotation, manual peer management)

Criterion	Tailscale	Headscale	Netbird (self-hosted)	Plain WireGuard
Cost	Free tier then paid for more devices/features	Free (self-hosted)	Free (self-hosted)	Free

I.3 Selection though process

Why plain WireGuard was rejected despite being "most trustable": plain WireGuard is the most minimal, most auditable option - literally just the protocol, nothing else to trust. But it fails requirement (4) outright as device count grows: every new peer needs manual key generation, manual IP allocation, manual `[Peer]` block distribution to every other peer that needs to reach it (full mesh = $O(n^2)$ config edits), no built-in NAT traversal (meaning port-forwarding headaches per device, worse for a phone on cellular which has no fixed forwardable IP at all), and no revocation/audit UI. For "reduce effort to setup and maintain" plus a phone-on-cellular requirement, it was ruled out as impractical beyond a trivial 2-device setup.

Why Tailscale was not chosen as primary: best UX by far, and would have won outright if "reduce effort" had been the top priority. Rejected as the primary pick because requirement (1)+(2)+(3) (secure your own devices/network, maximum visibility, trustable code end-to-end) are weakened by the control plane being an opaque third-party SaaS: Tailscale's own servers decide route distribution and policy enforcement, and you cannot inspect, audit, or run that part of the system yourself. This isn't a security *flaw* in Tailscale, but it directly conflicts with the stated priority ordering.

Why Headscale was not chosen: solves the self-hosting gap, but at the cost of running an **unofficial, reverse-engineered** reimplementation of a private, evolving protocol that Tailscale doesn't document or guarantee compatibility for. This directly conflicts with requirement (3) "trustable code end-to-end" - the thing enforcing your access control is a best-effort clone of someone else's closed system, not a system whose actual authors test and maintain the full stack together.

Why Netbird (self-hosted) was the final choice: the only option where **every single component** (client, management/control server, signal server, relay, dashboard) is open source **and** officially designed/supported to be run entirely by you, with WireGuard underneath (same trustable crypto core as the others), plus a real groups/ACL/DNS model so it doesn't regress to plain-WireGuard's manual-everything problem. Setup/maintenance effort is higher than Tailscale (no managed coordination server, no one-command HTTPS publish) but far lower than plain WireGuard (built-in NAT traversal/relay, dashboard-driven groups and policies, private DNS). Given the stated priority order (security/visibility/trust first, convenience second), this was the correct trade-off.

iPhone requirement - how it's satisfied: NetBird has an official iOS app (App Store), installable on the iPhone like any other VPN-style app: sign in via the same SSO flow as any other peer, and the phone becomes a normal mesh peer with its own NetBird IP, groups, and policies (see Part III, "peer-to-peer" decision). Once connected (even on cellular, no Wi-Fi needed), the phone can resolve the Custom Zone hostnames (e.g. `bigdude.mydomain.cloud`) and reach Timon/the hub exactly as if on the LAN, subject to whatever access policy scopes it to (e.g. the `iphone` group only reaching `timon-hosts`/`hub`, not the whole mesh).

I.4 Security vs comfort - hosting the NetBird controller as a VM *on* the Freebox

Two architectural options were weighed for where the self-hosted NetBird control plane (management server, signal server, relay, dashboard) should live:

- **(a) A VM hosted *on* the Freebox itself** (Freebox OS's built-in VM feature) - simplest, reuses hardware you already run 24/7, reuses the Freebox's own port-forwarding for the public ports NetBird needs (80/443/3478).
- **(b) A separate machine hosted *behind* the Freebox**, i.e. a distinct physical/virtual server on the LAN, with the Freebox just forwarding ports to it.

Security consideration: a compromise of the Freebox's own hypervisor layer could in theory affect VMs running on it, and (a) puts the control plane for your entire secure-access mesh on the same physical box as your internet gateway. Comfort/practicality consideration: the Freebox is **already** the single always-on, always-patched, always-powered device in the home - introducing a second physical always-on machine purely to host the controller (option b) adds a new piece of hardware to buy, power, maintain and patch, without actually removing the dependency on the Freebox being up (since it's still the only path in/out of the LAN either way).

Decision: **(a) - host the controller as a VM on the Freebox.** Rationale accepted: since the Freebox being down already means total loss of internet/remote-access regardless of where the controller VM physically lives, option (b) doesn't actually remove a single point of failure - it just adds a second piece of always-on hardware for no resilience gain. The marginal extra risk of hypervisor-level compromise on the Freebox was judged acceptable, given Freebox OS's VM feature is a mainstream, maintained product feature (not a DIY hypervisor), and the alternative provides no real mitigation for the actual dependency (the Freebox itself). This was explicitly confirmed as "an acceptable convenience decision" during the original brainstorming.

All actionable VM creation / Ubuntu configuration steps implementing this decision are in **Part II** below (not duplicated here).

I.5 Setup steps overview (what follows, at a glance)

Part II - NetBird Controller setup (Ubuntu VM on the Freebox)

II.1 Infrastructure requirements (official)

- A Linux VM: **1 vCPU, 2GB RAM** minimum.
- VM must be **publicly reachable** on **TCP 80 and 443**, and **UDP 3478** (STUN/relay).
- A **public domain** resolving to the VM's public IP - in this setup: `netbird.mydomain.cloud` (OVH-hosted zone).

II.2 Create the VM on the Freebox

1. Freebox OS -> **VM** section -> create a new VM: Ubuntu Server (latest LTS), 1+ vCPU, 2GB+ RAM, a dedicated disk (10-20GB is plenty).
2. Give the VM a **static DHCP lease** on the Freebox (so port-forwarding rules don't break if its LAN IP changes).

II.3 Networking / port-forwarding

- Freebox OS -> Port forwarding: forward WAN **80/tcp**, **443/tcp**, **3478/udp** to the VM's static LAN IP.
- Verify the Freebox is not behind **CGNAT** first (compare the Freebox's reported WAN IP to what an external "what's my IP" check sees from outside the home network) - if they differ, port-forwarding

won't work and a different reachability strategy is needed before proceeding.

II.4 DNS (OVH zone)

In the OVH DNS zone for `mydomain.cloud`, add an **A record** `netbird` -> the Freebox's public IP. Since Freebox WAN IP can change (unless you have a fixed IP option), either:

- Use Freebox's built-in **DynDNS** setting pointed at Free's own hostname, then add a **CNAME** `netbird.mydomain.cloud` -> that Free hostname in OVH, or
- Manually update the OVH A record if the IP changes rarely in practice.

This base `A netbird` record must already exist before running the quickstart script (§II.6). If the optional **NetBird Proxy service** is enabled during that script, also add a wildcard entry:

Type	Name	Content	Notes
A	`netbird`	Freebox public IP (or DynDNS target)	must already exist before running the script
CNAME	`*.netbird`	`netbird.mydomain.cloud`	only needed if the Proxy service is enabled

II.5 Ubuntu OS hardening and baseline setup

- **Automatic security updates**: enable unattended security-patch installation - important since this VM is directly internet-facing.

```
sudo apt install unattended-upgrades

sudo dpkg-reconfigure --priority=low unattended-upgrades
```

- **SSH hardening**: disable password auth, key-based only (`PasswordAuthentication no` in `/etc/ssh/sshd_config`), consider changing the default port or at minimum fail2ban/rate-limiting if left on 22. **Restrict SSH to the local LAN subnet at the `ufw` level** (below) rather than leaving it open to anywhere reachable - SSH is never port-forwarded on the Freebox (§II.3), so this closes the remaining gap of any device on the LAN being able to reach it, not just admin devices. Once the mesh is actually up, you can additionally restrict SSH to only be reachable over NetBird itself (chicken-and-egg: keep the Freebox VM console as an out-of-band fallback for the very first setup, before NetBird exists to tunnel through).

- **Firewall (ufw)**: default-deny inbound, explicitly allow only what's needed:

```
sudo ufw default deny incoming

sudo ufw default allow outgoing

sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp   ## SSH, LAN-only - replace wit

sudo ufw allow 80/tcp      ## ACME HTTP-01 / redirect

sudo ufw allow 443/tcp     ## Dashboard, API, Proxy (Traefik)

sudo ufw allow 3478/udp    ## STUN/relay

sudo ufw enable
```

Verify with `sudo ufw status numbered`: the SSH rule should show the subnet, not `Anywhere`. Optionally narrow further to a single admin IP, e.g. `sudo ufw allow from 192.168.1.50 to any port 22 proto tcp`. Test from a device outside the LAN (e.g. phone on cellular, NetBird disconnected) that `ssh` to the controller's LAN IP now times out/refuses, while a LAN device still connects.

- **Docker install**: the self-hosting quickstart script needs Docker + the Compose plugin, plus `jq` and `curl` as utility dependencies.

```
sudo apt install -y docker.io docker-compose-plugin

sudo apt install -y jq curl
```

- **Time sync**: ensure `systemd-timesyncd` (default on Ubuntu) is active - accurate clocks matter for TLS cert validity and ACME, and more generally for any token/code expiry checks across the mesh.
- **Swap** (optional but sensible on a 2GB VM): a small swapfile (1-2GB) as a safety margin for the Docker stack under memory pressure:

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo

echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

- **Backups**: back up the generated `config.yaml`, `docker-compose.yml`, `dashboard.env`/`proxy.env`, and the Docker volumes (this is the entire NetBird account database: users, peers, groups, policies) - this VM is the single source of truth for the whole mesh's identity and access control once set up.

II.6 Run the quickstart script

After II.5, download and run NetBird's official self-hosting installer (copy the **exact** current command from the live docs page at install time - it is versioned and changes): https://docs.netbird.io/selfhosted/selfhosted-quickstart.

During the script:

- **Reverse proxy selection**: choose option **`[0]` (Traefik)** - bundled, handles Let's Encrypt automatically, zero extra config. (Do **not** confuse this with the Caddy reverse proxy used later for Timon itself - that one runs on each Windows machine, not on the NetBird controller.)
- **Enable the NetBird Proxy service**: optional add-on that exposes per-service subdomains (`*.netbird.mydomain.cloud`) - if enabled, the wildcard `CNAME` DNS entry from §II.4 must already be in place.
- **CrowdSec** (optional): offered alongside the proxy for community IP-reputation blocking - worth enabling for a controller exposed to the public internet.
- Why the Default policy must be disabled immediately, not "left and restricted later"

- The Default policy (`All` -> `All`, bidirectional) makes the mesh **fail open**: every peer that joins can already reach every other peer, with no explicit decision made for that pairing.
- Two options were weighed:
  - **(a) Leave Default active after initial setup, add stricter policies later.** Convenient short-term, but every device enrolled in the meantime (including a quick test device, a one-off, a guest's phone during setup troubleshooting) is **immediately and fully exposed** to every other peer with no review step. There is no "grace window" - exposure starts the instant a peer authorizes.
  - **(b) Delete/disable Default immediately after enrolling the core devices, before adding anything else.** Short extra effort (a few groups + a couple of narrow policies, §III.3), but means the network **fails closed** from that point on: a newly joined peer has zero reachability until a human deliberately writes a policy granting it something. This matches the original requirement ("secure at most my devices and network") far better than deferred hardening - "I'll restrict it later" is a common source of forgotten exposure in home-lab setups, since there's rarely a forcing function to go back and do it once things "just work."
- **Decision**: (b) - the Default policy was deleted right after the first three devices (`bigdude`, hub, iPhone) were grouped, and before any further peer was added to the account. Going forward, every new peer requires an explicit group assignment and policy before it can reach anything, by construction.

Generated files: `docker-compose.yml`, `config.yaml`, `dashboard.env`, and (if proxy enabled) `proxy.env`. Manage the stack afterwards with `docker compose` commands from that directory.

II.7 First login

1. Browse to `https://netbird.mydomain.cloud` - redirects to `/setup` (only reachable while no users exist yet).
2. Create the admin account: email, name, password.
3. Log in normally afterwards.

Part III - NetBird Groups and Access Policies

III.1 Why Peer-to-peer network, not "remote access network" (Networks/routing-peer model)

NetBird offers two distinct ways to reach a device:

- **Direct peer-to-peer**: the target device itself runs the NetBird client and becomes a full mesh peer - reachable directly (or via relay/STUN if direct UDP fails), with its own identity, groups, and policies.
- **Networks / routing-peer model**: used for devices that *can't* run a NetBird client (embedded devices, IoT, legacy boxes) - another peer on the same LAN acts as a routing gateway, advertising a Resource (an IP/CIDR/domain) on their behalf. Traffic is routed through that gateway peer rather than reaching the target peer's own WireGuard identity directly.

Why P2P was chosen for `bigdude` (Timon host) and the home-automation hub:

- **Both can run a full NetBird client directly** - no embedded/legacy constraint forcing the routing-peer workaround.

- **Smaller trust surface**: a routing peer becomes a single chokepoint that can see/relay traffic for every resource behind it; direct P2P means each device enforces its own policy independently, with no intermediary peer to compromise.
- **Simpler policy model**: P2P groups/policies target the device itself (one group membership, one set of rules); the Networks model adds an extra layer (resource definition + routing peer assignment + policy on the resource) for no benefit here, since there's no non-client-capable device requiring it.
- **No extra hop / extra latency**: direct P2P (or relay-assisted P2P) is one WireGuard hop; routing-peer traffic adds a second hop through the gateway peer's LAN.

When the Networks/routing-peer model would be the right call instead: a smart-home device that truly cannot install a NetBird client (e.g. a sensor, a proprietary hub with no package support) - then a single `/32` Resource pointed at just that device's LAN IP, routed through one trusted gateway peer, scoped to the narrowest possible access policy, is the appropriate tool. Reserved for that case only; not used as the default.

III.2 Core concepts (official)

- **Groups**: named sets of peers (and optionally resources), similar to tags. A peer can belong to multiple groups. There's a built-in **`All`** group auto-containing every peer - it cannot be deleted or renamed.
- **Policies**: ALLOW rules only (no explicit deny/priority system) - a connection is permitted only if some policy's Source/Destination groups match both ends; everything else is denied by default. Each policy is **bidirectional** or **unidirectional** (only the source side can initiate).
- **Default policy**: auto-created on account creation, using `All` -> `All`, bidirectional - i.e. full mesh, everyone can reach everyone. This is intentionally permissive to make initial setup painless.

Official reference: [Managing Access with NetBird: Groups and Access Policies].

III.3 Recommended approach for this setup

1. **Enroll first**: add the core devices as peers before touching policies - `bigdude` (Timon host), the home-automation hub, and your iPhone. With the Default policy still active they can all already reach each other, which is fine for initial verification.
2. **Create custom groups**, e.g.:
   - `timon-hosts` - any machine running Timon (e.g. `bigdude`).
   - `hub` - the home-automation hub.
   - `iphone` - your personal phone.
   - `timon-admins` - devices allowed to browse/administer Timon remotely (likely just `iphone` plus any other personal device).
3. **Delete (or disable) the Default `All->All` policy** (if not already done) once the core devices are enrolled and grouped - do this *before* adding any further/future devices, so nothing new is implicitly granted full access.
4. **Create narrow policies**, one per real need, e.g.:
   - `iphone` -> `timon-hosts`, TCP 443 only, unidirectional (phone initiates, host doesn't need to reach back).
   - `iphone` -> `hub`, whatever port(s) the hub's own UI/API needs, unidirectional.
   - Avoid `All`-group policies entirely going forward; always name explicit source/destination groups.

5. When adding a private DNS name (Custom Zone) for a service, scope the zone to the specific **Distribution Group** that needs it (e.g. `timon-admins`), not to `All`.

Strengthening rules further: the policies above control *which groups* can reach each other, but NetBird can also condition access on the *state of the connecting device* via Posture Checks - e.g. requiring a minimum NetBird client version, restricting by country/region, allow/deny-listing peer network ranges, requiring specific running processes, or an OS-issued certificate. Attaching a posture check to the `iphone` -> `timon-hosts` policy (for example, a minimum client version) is a natural next hardening step once the base groups/policies here are working.

III.4 Install and enroll NetBird (Timon hosts on Windows)

1. Download the NetBird **EXE** or **MSI** installer from `https://pkgs.netbird.io/windows/x64` (or `/windows/msi/x64`). Official docs: https://docs.netbird.io/get-started/install/windows.
2. Run it - installs to `C:\Program Files\NetBird` and registers the NetBird daemon as its own Windows service automatically.
3. Open a **new** terminal (PATH only updates for new terminals):

   ```
   netbird up --management-url https://netbird.mydomain.cloud
   ```

   Follow the browser SSO link to authorize the peer. If the device-code expires (a few minutes), just re-run `netbird down` then `netbird up ...` again to get a fresh one.
4. Confirm the assigned private IP:

   ```
   netbird status
   ```

   Example used throughout this doc for the `bigdude` machine: NetBird IP `100.64.108.163`.
5. In the NetBird Dashboard, add this peer to the `timon-hosts` group (above).
6. Test reachability over cellular (Wi-Fi off, NetBird toggled on in the app): browse to `https://bigdude.mydomain.cloud` (Part IV, §IV.5) - should load Timon's sign-in page exactly as from a laptop on the mesh.

III.5 Install and enroll NetBird on the iPhone

1. Install **NetBird** from the App Store (official app, published by `netbird.io`).
2. Open the app -> tap **Add Profile** (or the equivalent first-run prompt) -> enter the management URL: `https://netbird.mydomain.cloud` (same self-hosted controller as the Windows peers).
3. Tap **Connect** / **Sign in** - this opens the same browser-based SSO login used for every other peer. Log in with your NetBird account and approve the authorization request.
4. iOS will show a system prompt to **Add VPN Configurations** - tap **Allow** (iOS implements NetBird's WireGuard tunnel as a standard VPN profile; this is expected and required for the tunnel to work).
5. Once connected, the app shows **Connected** with the phone's assigned NetBird IP - confirm it in the NetBird Dashboard's Peers list as well.

6. In the NetBird Dashboard, add this peer to the `iphone` group (above) - this is what the `iphone` -> `timon-hosts` / `iphone` -> `hub` policies (§III.3) target.
7. Test reachability over cellular (Wi-Fi off, NetBird toggled on in the app): browse to `https://bigdude.mydomain.cloud` (Part IV, §IV.5) - should load Timon's sign-in page exactly as from a laptop on the mesh.

III.6 NetBird Custom DNS Zone

Steps to create the zone, add the hosts, and scope distribution:

1. NetBird Dashboard -> **DNS** -> **Nameservers/Zones** (depending on dashboard version) -> **Add Zone**.
2. **Zone name/domain**: enter the base domain used for all entries in this zone, e.g. `mydomain.cloud` (must match the OVH-hosted domain, §II.4 / §IV.3).
3. **Add Record** (one per hostname), for each of the two entries above:
   - **Name**: the full hostname, e.g. `bigdude.mydomain.cloud`.
   - **Target/IP**: the peer's NetBird IP (e.g. `100.64.108.163`) - find it via `netbird status` on that machine or the Dashboard's Peers list.
   - Repeat for `bigdude-daemon.mydomain.cloud`, same target IP (same peer, different Caddy site block, §IV.4a).
4. **Distribution group(s)**: on the zone (or per-record, depending on dashboard version), set this to `timon-admins` (or your actual equivalent group, e.g. `Admins`) - **not** `All`. Only peers whose group is listed here receive this DNS record at all; everyone else gets no resolution for it, by design.
5. Save, then from an already-enrolled peer in that distribution group, confirm resolution: `nslookup bigdude.mydomain.cloud` (or `ping`) should return the expected NetBird IP within a minute or two of saving.
6. Whenever a group is renamed or restructured (e.g. splitting a group, adding new admin devices), **revisit this zone's distribution group list** - it does not update automatically, and a stale list silently breaks resolution only for the newly-affected peers (see troubleshooting note below).

Both blocks reuse the same OVH DNS-01 credentials - Caddy requests and renews a separate certificate per hostname automatically (Part IV, §IV.4). Confirmed working end-to-end: Caddy logs show `challenge_type: dns-01`, `authz_status: valid`, and `certificate obtained successfully` for `bigdude.mydomain.cloud` with no errors (first-run "cannot find path" messages are benign - just Caddy creating its local ACME account file).

TLS summary: browser/iPhone <-> Caddy is HTTPS with a real, publicly-trusted Let's Encrypt cert (no warnings); Caddy <-> Timon (`127.0.0.1:3110` / `127.0.0.1:3010`) is plain HTTP over loopback only, which never leaves the machine.

Troubleshooting `ERR_NAME_NOT_RESOLVED` on the custom zone hostname: unlike the automatic per-peer name (`<peer-name>.netbird.selfhosted`), a Custom Zone record is only pushed to peers whose group is listed in that zone entry's **Distribution group** field - it is not a mesh-wide DNS record. If a peer (e.g. the iPhone) can resolve its own automatic `.netbird.selfhosted` name but gets `ERR_NAME_NOT_RESOLVED` specifically on `bigdude.mydomain.cloud`, the fix is: NetBird Dashboard -> DNS -> Zones -> the zone entry -> confirm the peer's actual group (e.g. `Admins`) is listed under Distribution group - not just the group(s) the zone was originally scoped to. A group rename or restructuring (e.g. splitting `Admins` out of an older group name) can leave the zone's distribution list stale, silently breaking resolution for the affected peer only (toggling the NetBird connection off/on forces a fresh config pull once the group is fixed).

Official reference: DNS in NetBird (Nameservers, Custom Zones, Match/Search Domains).

Part IV - Timon and Windows reachable via NetBird

Non-negotiables

- Timon's two listeners - **web app** (`app.listen`, default `127.0.0.1:3100`) and **daemon API** (`daemon.listen`, default `127.0.0.1:3000`) - stay on `127.0.0.1` forever. Never bind to `0.0.0.0` or a network interface IP directly.
- Create the administrator account first (local access or SSH tunnel), before opening anything else.
- Once a reverse proxy sits in front, set `daemon.behindTlsProxy: true` and restart.
- Never set `daemon.auth` to `"off"`.
- NetBird has **no automatic HTTPS** like Tailscale's `tailscale serve` - a manual reverse proxy (Caddy) with a real certificate is required.

IV.1 Windows background-run notes

- Timon runs via the **`TimonOrchestrator` scheduled task** (not a true service), started at boot, surviving sign-out - but **waiting for your first sign-in after every restart**. For an always-reachable machine: enable automatic sign-in (`netplwiz` -> uncheck "Users must enter a user name and password"), or sign in manually after each reboot. Also disable sleep: Settings -> System -> Power -> "When plugged in, put my device to sleep after" -> **Never**.
- Config file: `%USERPROFILE%\.timon\timon.config.json`.
- Service control: `timon service status` / `timon service stop` / `timon service start`.
- Service health/restart events, port status, and startup warnings are visible in the desktop log (`%USERPROFILE%\.timon\tmp\daemon-<date>.log`) - a "did not answer within 120s" or `ERR_CONNECTION_REFUSED` warning simply means the daemon/app hadn't finished starting yet; wait for the health check to report OK before troubleshooting further (can take up to ~180s after a service restart).

IV.2 Keep Timon's listeners on loopback

Confirm in `%USERPROFILE%\.timon\timon.config.json` (actual values on `bigdude`: app on `3110`, daemon on `3010` - non-default, confirm against your own install):

```
{
  "app": { "listen": { "host": "127.0.0.1", "port": 3110 } },
  "daemon": { "listen": { "host": "127.0.0.1", "port": 3010 }, "behindTlsProxy": true }
}
```

IV.3 OVH API credentials setup for DNS-01

The `caddy-dns/ovh` plugin needs four values: an **endpoint** name and three secrets (**application key**, **application secret**, **consumer key**).

These credentials are scoped to the OVH zone/account, not to any one machine. The same `application_key`/`application_secret`/`consumer_key` can (and should, for simplicity) be reused in every machine's Caddyfile `dns ovh` block - what they authorize is "permission to edit DNS records in

this zone," which any Caddy instance needs regardless of which machine it runs on. Each machine still gets its own certificate for its own hostname; only the credential is shared. A separate token per machine is optional, only useful for easier per-machine revocation later. If a machine's Caddyfile is missing these values (or has placeholders), Caddy's log shows `missing authentication information, you need to provide one of the following: application_key/application_secret, client_id/client_secret, or access_token` - that error means the credentials below were never actually set on that machine.

Steps:

1. Go to OVH's API credential creation page for your account's region - for most European `.cloud`/`.com` domains managed under the EU dashboard, this is `https://eu.api.ovh.com/createToken/` (use `ca.api.ovh.com` or `us.api.ovh.com` instead if your OVH account is Canada/US-based - this also determines the `endpoint` value in the Caddyfile: `ovh-eu`, `ovh-ca`, or `ovh-us`).

2. Fill in:

   - **Application name**: e.g. `caddy-dns-mydomain`
   - **Application description**: e.g. `Caddy DNS-01 for mydomain.cloud`
   - **Validity**: unlimited (or set a renewal reminder if you prefer time-boxing it)

3. Grant exactly these API rights, scoped to the zone (replace `mydomain.cloud` with your real domain - narrower is better than the wildcard `*` scope):

   ```
   GET    /domain/zone/mydomain.cloud/record
   POST   /domain/zone/mydomain.cloud/record
   GET    /domain/zone/mydomain.cloud/record/*
   PUT    /domain/zone/mydomain.cloud/record/*
   DELETE /domain/zone/mydomain.cloud/record/*
   GET    /domain/zone/mydomain.cloud/soa
   POST   /domain/zone/mydomain.cloud/refresh
   ```

4. Submit - OVH immediately returns an **Application Key** and **Application Secret**. Save both now; the secret is shown only once. Generating the **Consumer Key** (binds the application key to your account with the rights above) requires one authenticated call to OVH's API - the create-token web form above typically already returns all three values together; if using the raw API flow instead, use `ovh/go-ovh`'s helper or OVH's own "Create consumer key" quickstart with the same endpoint and application key.

5. Store all three values (`application_key`, `application_secret`, `consumer_key`) plus the `endpoint` (`ovh-eu`) securely - e.g. as Windows environment variables referenced from the Caddyfile instead of hardcoding them in plaintext:

   ```
   [System.Environment]::SetEnvironmentVariable("OVH_ENDPOINT", "ovh-eu", "Machine")
   [System.Environment]::SetEnvironmentVariable("OVH_APPLICATION_KEY", "...", "Machine")
   [System.Environment]::SetEnvironmentVariable("OVH_APPLICATION_SECRET", "...", "Machine")
   [System.Environment]::SetEnvironmentVariable("OVH_CONSUMER_KEY", "...", "Machine")
   ```

The variables will be referenced in the Caddyfile with `{$OVH_ENDPOINT}`, `{$OVH_APPLICATION_KEY}`, etc. instead of inline strings - keeps secrets out of the Caddyfile itself (restart the `Caddy` service after setting machine-level env vars so it picks them up).

IV.4 Install Caddy as a Windows service with `sc.exe` (built into Windows, no extra tool)

Caddy needs a custom build (OVH DNS plugin, §IV.4a) and OVH API credentials (§IV.3) before it can be registered as a service with DNS-01 TLS - both are covered here, followed by the actual service registration (§IV.4c).

IV.4a Building a custom Caddy with the OVH DNS plugin (`xcaddy`)

Stock Caddy binaries (from caddyserver.com/download) don't include third-party DNS provider plugins like `caddy-dns/ovh` - they must be compiled in via `xcaddy`. This is built **directly on the Windows machine**, in PowerShell, with no Linux/WSL step needed.

1. Install Go for Windows (one-time): download and run the installer from https://go.dev/dl/ (pick the `.msi` for `windows-amd64`), then open a **new** PowerShell window so `PATH` picks up `go.exe`. Confirm:

   ```
   go version
   ```

2. Install `xcaddy`:

   ```
   go install github.com/caddyserver/xcaddy/cmd/xcaddy@latest

   $env:PATH += ";$(go env GOPATH)\bin"
   ```

   (Add `$(go env GOPATH)\bin` to your permanent `PATH` via System Properties -> Environment Variables, so this doesn't need repeating in every new session.)

3. Build `caddy.exe` with the OVH plugin:

   ```
   xcaddy build --with github.com/caddy-dns/ovh
   ```

   This produces `caddy.exe` in the current directory - `xcaddy` fetches Caddy's source plus the named plugin module and compiles them together; no manual Go coding needed.

4. Verify the plugin is present before shipping it:

   ```
   .\caddy.exe list-modules | Select-String ovh
   ```

   Expect to see `dns.providers.ovh` in the output.

5. Move the built binary into place:

   ```
   New-Item -ItemType Directory -Force -Path C:\Tools\caddy | Out-Null

   Move-Item -Force .\caddy.exe C:\Tools\caddy\caddy.exe
   ```

6. Whenever Caddy itself needs a version bump later, repeat steps 3-5 with the newer Caddy/plugin versions - `xcaddy build` always pulls latest unless a version is pinned (e.g. `xcaddy build v2.8.4 --with github.com/caddy-dns/ovh`); then re-run `sc.exe stop Caddy` / `sc.exe start Caddy` (§IV.4c) so the already-registered service picks up the new binary.

Caddyfile sample (split app/daemon, using env vars):

```
bigdude.mydomain.cloud {
    bind 101.63.99.123
    tls {
        dns ovh {
            endpoint {$OVH_ENDPOINT}
            application_key {$OVH_APPLICATION_KEY}
            application_secret {$OVH_APPLICATION_SECRET}
            consumer_key {$OVH_CONSUMER_KEY}
        }
    }
    log {
        output file C:\Tools\caddy\logs\app.log
    }
    reverse_proxy 127.0.0.1:3110
}

bigdude-daemon.mydomain.cloud {
    bind 101.63.99.123
    tls {
        dns ovh {
            endpoint {$OVH_ENDPOINT}
            application_key {$OVH_APPLICATION_KEY}
            application_secret {$OVH_APPLICATION_SECRET}
            consumer_key {$OVH_CONSUMER_KEY}
        }
    }
    log {
        output file C:\Tools\caddy\logs\daemon.log
    }
    reverse_proxy 127.0.0.1:3010
}
```

Test the Caddyfile before applying it (from `C:\Tools\caddy`):

```
.\caddy.exe validate .\Caddyfile

.\caddy.exe fmt --overwrite .\Caddyfile
```

`validate` catches syntax/config errors without starting the server; `fmt --overwrite` reformats the file to Caddy's canonical style in place (useful after manual edits, and safe to run before every `sc.exe stop Caddy` / `sc.exe start Caddy` cycle).

IV.4c Register Caddy as a Windows service

1. Confirm the custom-built `caddy.exe` (§IV.4a) is at `C:\Tools\caddy\caddy.exe`, and the OVH environment variables (§IV.3) are set.

2. Register and start the service:

```
sc.exe create Caddy start= auto binPath= "C:\Tools\caddy\caddy.exe run --config C:\Tools\cad

sc.exe start Caddy
```

   Stop / remove if ever needed:

```
sc.exe stop Caddy

sc.exe delete Caddy
```

3. Optional: auto-restart on crash (`sc.exe` doesn't do this by default):

```
sc.exe failure Caddy reset= 0 actions= restart/5000
```

4. Config changes need a manual restart - `sc.exe`-registered services can't be signaled to reload gracefully:

```
sc.exe stop Caddy

sc.exe start Caddy
```

5. Firewall: scope the Windows Defender Firewall allow rule to the NetBird virtual adapter/category (set that adapter's network category to **Private** when Windows first prompts), not Public/Private broadly. Caddy's `bind` directive already prevents listening on the public adapter at all.

6. Logs: Caddy with `sc.exe` writes to stdout/stderr by default, which goes nowhere without a console attached - add an explicit `log` directive per site block to persist logs to a file (see Caddyfile in §IV.4a), or check Event Viewer -> Windows Logs -> Application as a fallback.

Running `caddy.exe run --config ...` interactively in a terminal also shows live output for troubleshooting - but **stop the service first** (`sc.exe stop Caddy`). Caddy's own admin API binds `127.0.0.1:2019` by default, and only one Caddy process can hold that port at a time; running the interactive command while the service is still up fails with `listen tcp 127.0.0.1:2019: bind: Only one usage of each socket address...`. Restart the service (`sc.exe start Caddy`) once done debugging.

Staging vs production ACME CA: while debugging TLS issuance, Caddy's log may show requests going to `acme-staging-v02.api.letsencrypt.org` (Let's Encrypt's staging environment - issues untrusted test certificates, used to avoid production rate limits while troubleshooting). Once DNS-01 issuance succeeds cleanly against staging, remove/adjust any `acme_ca` staging override in the Caddyfile so it issues from the real production CA (`acme-v02.api.letsencrypt.org`) - otherwise browsers will keep showing an untrusted-certificate warning indefinitely.

IV.4d Why two hostnames (app vs daemon) - and why it matters for pairing

Rule: pairing always targets the daemon hostname. In the Machines plugin, enter `https://<host>-daemon.mydomain.cloud` (Caddy -> `127.0.0.1:3010`), never the bare

`https://<host>.mydomain.cloud` (Caddy -> web app `127.0.0.1:3110`). The bare hostname is for browsing only.

Device pairing (Settings -> Access & network -> Devices, used by the Machines plugin) calls the **daemon's** API (`/api/devices/redeem`), not the web app. If the reverse proxy points at the web app port instead, the web app answers **401 Unauthorized**, and Timon surfaces this as a misleading **"wrong, expired or already used"** pairing-code error - even though the code itself is still valid. Tell-tale sign: the code is still shown with time remaining when the error appears, proving the request never reached the daemon.

Two situations both produce this exact misleading message:

1. Proxy points to the **web app** (3110) instead of the **daemon** (3010).
2. The proxy itself requires its own authentication (basic auth, Authelia, Authentik, etc.) in front of `/api/*` - its 401 is returned before Timon is ever reached.

Also: self-signed certs (`tls internal`) are **rejected outright** for pairing - only a real, CA-trusted certificate works (which is exactly what the DNS-01/OVH setup below provides). The **"Make this machine reachable"** button in Timon's UI only works with Tailscale; it's a no-op for NetBird.

Diagnostic command

From any machine, using the exact HTTPS address entered in Machines:

```
curl.exe -s -X POST https://YOUR-ADDRESS/api/devices/redeem -H "content-type: application/json"
```

- `{"error":"unauthorized"}` -> the web app answered, not the daemon -> fix the proxy target.
- A login page / proxy-branded response -> the proxy's own auth is in the way -> remove auth on `/api/*`.
- `{"error":"invalid_request"}` or `device_requires_tls` -> request reached the daemon correctly -> look elsewhere.

Conclusion: the web app and the daemon must be served on **separate hostnames**, each with its own clean `reverse_proxy` target - not combined on one hostname, and not split by path (fragile, risks misrouting).

IV.5 Apply and use

```
sc.exe stop Caddy

sc.exe start Caddy
```

- **Day-to-day browsing**: `https://bigdude.mydomain.cloud`
- **Pairing a new device / Machines plugin remote address**: `https://bigdude-daemon.mydomain.cloud` (the **daemon** hostname, never the bare app hostname). Steps:
  1. On the machine to add (`bigdude`), generate a fresh pairing code: Settings -> Access & network -> Devices. The code must come from this machine, not from the one you pair from.
  2. On the machine you pair from, open Machines and enter the address `https://bigdude-daemon.mydomain.cloud` and the code.

3. Do it right away. Generate a new code after any restart of `bigdude`'s Timon service, and after any failed attempt.
4. If you get "wrong, expired or already used", run the diagnostic command in §IV.4d against the daemon hostname before you try a new code.

IV.6 Verify

- `curl.exe -s https://bigdude.mydomain.cloud` from an enrolled remote peer - should return a redirect to `/login`. (Do not add `-UseBasicParsing`: it is a PowerShell `Invoke-WebRequest` parameter, and `curl.exe` reads it as `-U`, a proxy user.)
- Confirm the hostnames are not swapped, from an enrolled remote peer:

```
curl.exe -s -D - -o NUL https://bigdude.mydomain.cloud

curl.exe -s -D - -o NUL https://bigdude-daemon.mydomain.cloud
```

The bare hostname must return `307` to `/login` (web app). The `-daemon` hostname must return `401` with `X-Powered-By: Express` (daemon). If they are the other way round, swap the `reverse_proxy` targets in the Caddyfile and restart Caddy.

- `sc.exe query Caddy` -> `RUNNING` after reboot (service is `start= auto`).
- Confirm both hostnames are unreachable from a device NOT enrolled in NetBird.
- Confirm `127.0.0.1:3110` and `127.0.0.1:3010` both answer locally before attempting remote pairing (avoids the false "code refused" symptom caused by the app/daemon still starting up).

IV.7 Access control reminder

Both hostnames resolve to the same NetBird peer IP and port 443 - scope the NetBird access policy once: source group `timon-admins` -> this peer, port 443 only (Part III, §III.3 step 4). No separate policy is needed per hostname, since routing between them is handled by Caddy's SNI-based site blocks, not by NetBird.

Conclusion: Architecture overview

Reading the diagram:

- **Step 1 (thin arrows, to the controller)**: every mesh peer - laptop, iPhone, `bigdude`, the hub - authenticates to the self-hosted **NetBird controller VM** (on the Freebox) via SSO, and receives the current peer list, routes, and access policy from it. The controller never sits in the actual data path once peers know how to reach each other.
- **Step 2 (thick/dashed arrows, peer-to-peer)**: actual traffic between an external device and a LAN peer flows as a **direct WireGuard tunnel** between the two peers (NAT-traversed; only relayed through the controller's relay/TURN component as a fallback when direct UDP fails) - this is the peer-to-peer model chosen over the routing-peer/"Networks" model (Part III, §III.1).
- **Green box (onboarded)**: `bigdude` and the hub run the NetBird client themselves, so they're reachable individually, each governed by its own group/policy (Part III) - nothing is reachable by default (Default policy disabled, §III.3 step 3).
- **Red box (not onboarded)**: any LAN device without a NetBird client (NAS, printer, legacy IoT) is **only reachable from other devices already on the LAN** - remote peers have **no path to them at all** unless a routing-peer/Resource is deliberately configured for that specific device (the case

NetBird's "Networks" feature exists for, reserved and not used here by default).
- The Freebox itself only does **port-forwarding** (80/443/3478) to the controller VM - it is not in the data path for peer-to-peer traffic once peers have connected directly.

Why external traffic reaches LAN devices despite the Freebox blocking incoming connections

The Freebox blocking *unsolicited* incoming traffic doesn't prevent the WireGuard tunnel from forming, because of how NAT traversal (UDP hole punching) works - the connection is always **initiated from inside the LAN outward**, never accepted as an unsolicited inbound connection:

1. **The LAN device dials out first.** `bigdude` (or the hub), running the NetBird client, opens an **outbound** UDP connection to the controller/signal server. The Freebox's NAT/firewall always allows outbound traffic and the return replies on that same connection - this is normal, expected NAT behavior, not a port-forward.
2. **The signal server introduces both sides.** The external laptop/phone also connects outbound to the controller. The controller (via its signal component) exchanges each side's public IP:port (as seen from the outside, after NAT) and WireGuard public keys - it's matchmaking, not routing the data.
3. **Hole punching.** Both peers then send UDP packets toward the *other's* observed public address, simultaneously, from the connections they already have open outbound. Because the Freebox's NAT table now has an "expected return traffic" mapping for that external address:port (from step 1's outbound packet), the incoming WireGuard packets from the external peer are seen by the Freebox as *replies to a connection it opened itself* - so they're let through, with **no port-forward rule needed at all**.
4. **If hole punching fails** (strict/symmetric NAT, some mobile carriers), NetBird falls back to the **relay/TURN** component - which both sides are *already* connected to outbound (same as step 1), so again no inbound port is needed; traffic is relayed through a connection both sides initiated.

So the only port-forward actually configured on the Freebox (80/443/3478, Part II §II.3) is for the **controller VM itself** to be reachable (dashboard, ACME, STUN) - not for `bigdude` or the hub. Those LAN devices are reached purely via connections they themselves opened outward, which is exactly why no incoming-traffic rule is needed for them specifically.

Connection flow, step by step

The diagram below walks through the exact message flow when an external device (laptop or iPhone) tries to reach `bigdude` on the LAN, from the moment both clients are idle/connected to the moment traffic flows directly:

Key takeaway: the controller is only ever involved in **steps 1-2** (authenticate, authorize, exchange addresses). From **step 3** onward, the controller either steps out of the path entirely (direct P2P) or stays only as a passive relay (fallback) - it never initiates or accepts an inbound connection to the LAN on `bigdude`'s behalf, which is why no Freebox port-forward is required for `bigdude` itself.

Glossary

- **NetBird**: open-source, WireGuard-based mesh VPN platform with its own management server, signal server, relay/TURN, and dashboard; can be used as a fully managed cloud service or self-hosted end-to-end.

- **Tailscale**: commercial WireGuard-based mesh VPN product with a proprietary, SaaS-hosted control plane (coordination server); the client is open source, the control plane is not.
- **Headscale**: open-source, community-maintained, reverse-engineered reimplementation of Tailscale's control-plane protocol, letting official Tailscale clients connect to a self-hosted server instead of Tailscale's SaaS.
- **WireGuard**: the modern, minimal, cryptographically audited VPN protocol/kernel module underlying Tailscale, Headscale, and NetBird alike.
- **Control plane**: the component(s) responsible for authentication, peer discovery, key/route distribution, and access-policy decisions (in NetBird: management server + signal server + dashboard). Never carries actual application traffic.
- **Data plane**: the actual path application traffic takes between peers - in NetBird/WireGuard, a direct encrypted UDP tunnel between the two peers, bypassing the control plane whenever possible.
- **Management server**: the NetBird component that stores peers, groups, routes, and policies, and serves the dashboard/API.
- **Signal server**: the NetBird component peers use to exchange connection metadata (public keys, observed addresses) needed to establish a direct WireGuard tunnel.
- **Relay / TURN**: the NetBird component used as a fallback data path when two peers cannot establish a direct UDP tunnel (e.g. symmetric/strict NAT); still never requires an inbound port-forward on either peer.
- **NAT traversal / UDP hole punching**: the technique letting two peers behind NAT/firewalls (like the Freebox) establish a direct connection without either side needing an inbound port-forward, by both sending traffic outward toward each other's observed public address.
- **Peer**: any device running the NetBird client and enrolled in the mesh (laptop, iPhone, `bigdude`, the home-automation hub).
- **Group**: a named set of peers and/or users in NetBird, used as the source/destination unit in access policies.
- **Policy (ACL)**: a NetBird rule that allows (or, by omission, denies) traffic between a source group and a destination group, optionally scoped to specific ports/protocols.
- **Peer-to-peer network (routing mode)**: the NetBird mode where every onboarded device runs its own client and peers connect directly to one another - the mode chosen for this setup (see §III.1).
- **Routing peer / "Networks" (remote access network)**: NetBird's alternate mode where one peer acts as a gateway/router for other, non-client LAN devices and subnets - not used here by default, reserved for devices that cannot run a NetBird client.
- **Custom Zone**: NetBird's private DNS feature, used here to give enrolled peers resolvable hostnames (e.g. `bigdude.mydomain.cloud`) without exposing them in public DNS.
- **Caddy**: the open-source web server/reverse proxy used on `bigdude` (Windows) to terminate TLS and route the two published hostnames (app/daemon) to Timon's local ports.
- **DNS-01 challenge**: an ACME (Let's Encrypt) domain-validation method that proves domain ownership via a DNS TXT record rather than an HTTP request - used here because the OVH-managed domain's records can't be validated via inbound HTTP through the Freebox for every hostname.
- **xcaddy**: the official Caddy build tool used to compile a custom `caddy.exe` with third-party plugins (here, the `caddy-dns/ovh` DNS-01 provider) not included in stock Caddy builds.
- **Freebox**: the French home internet gateway/router (by Free) used here both as the home's internet access point and, via its built-in VM feature, as the host for the NetBird controller VM.